Blue X V1 Role Document

Mobile Frontend Developer

This document explains the mobile frontend responsibility for Version 1 of the Blue X payment verification app.

The mobile app is responsible for screens, user input, API calls, loading states, and displaying backend responses. The mobile app must not decide whether a payment is real. That decision belongs to the backend.

Main Goal

Build the mobile app experience for three main pages:

- Profile
- Scanner
- Dashboard

The app should be simple enough for a shop owner or staff member to use during a real sale.

General Frontend Rules

- Do not call external payment provider APIs directly.
- Do not store external provider API keys in the mobile app.
- Do not decide payment status in the frontend.
- Always show loading state while waiting for backend.
- Always show backend errors clearly.
- Use the authentication token for protected API calls.
- Keep the UI simple and readable.

Authentication Screens

The mobile app should include:

- Register screen
- Login screen

- Logged-in app navigation
- Logout action

Register should call:

```
POST /api/accounts/register/
```

Login should call:

```
POST /api/accounts/login/
```

After login/register:

- Store the returned token securely.
- Use the token for Profile, Scanner, and Dashboard APIs.
- If token is missing or invalid, send the user back to login.

Profile Page

The Profile page should show and update user/business information.

The page should display:

- Full name
- Phone number
- Business name
- Business type
- Receiver name
- Receiver account/phone
- Default provider/bank
- Scan usage if backend returns it

The page should allow:

- Editing business name
- Editing business type
- Editing receiver name
- Editing receiver account/phone
- Editing default provider/bank
- Logout

APIs:

```
GET /api/accounts/profile/
PUT /api/accounts/profile/
POST /api/accounts/logout/
```

Expected profile update request:

```
{
  "business_name": "Blue Shop",
  "business_type": "Retail",
  "receiver_name": "Natnael Example",
  "receiver_account": "1000017692643",
  "default_provider": "cbe"
}
```

Scanner Page

The Scanner page is the main app action.

For the first build, manual entry is required. QR scan can be added after manual entry works. OCR can come later.

Scanner inputs:

- Provider/bank
- Receipt ID or transaction reference
- Receipt URL if QR scan is available
- Amount if available
- Sender name if available
- Sender phone if available
- Receiver name/account from profile or manual override

API:

```
POST /api/verification/verify/
```

Request example:

```
{
  "provider": "cbe",
  "bank_name": "CBE",
  "receipt_id": "FT23001234ABC",
  "transaction_reference": "FT23001234ABC",
  "receipt_url": "",
```

```
  "sender_name": "Customer Name",
  "sender_phone": "0911223344",
  "amount": "500.00",
  "receiver_name": "Natnael Example",
  "receiver_account": "1000017692643"
}
```

The page must show:

- Loading state while request is running
- Success result card
- Pending result card
- Failed result card
- Unknown result card
- Clear error if backend rejects the request

Result card should display:

- Status
- Amount
- Provider/bank
- Sender name
- Receipt/reference ID
- Backend message
- Verification date/time

Dashboard Page

The Dashboard page shows financial summary and payment history.

APIs:

```
GET /api/verification/dashboard/summary/?period=week
GET /api/verification/dashboard/summary/?period=month
GET /api/verification/dashboard/summary/?period=year
GET /api/verification/transactions/
```

Dashboard should show:

- Total verified amount
- Total scans
- Verified count

- Pending count
- Failed count
- Unknown count
- Provider/bank breakdown
- Recent transactions
- Week/month/year filter

The dashboard should not be crowded. The first screen should show only the most important summary, then the user can scroll for details.

Done When

This role is complete for V1 when:

- User can register and login from mobile.
- Token is stored and used for protected requests.
- Profile page can load and update business profile.
- Scanner page can submit manual verification request.
- Scanner page shows loading and result card.
- Dashboard page loads summary and transaction history.
- Errors are displayed clearly.
- The app does not expose provider API keys or make provider calls directly.